Load Balancing Methods

In this lesson, you will get an insight into hardware and software load balancing.

We'll cover the following

- Hardware load balancers
- Software load balancers
- Algorithms/Traffic routing approaches leveraged by load balancers
  - Round robin and weighted round robin
  - Least connections
  - Random
  - Hash

There are primarily three modes of load balancing:

- DNS Load Balancing
- Hardware-based Load Balancing
- Software-based Load Balancing

We discussed *DNS load balancing* in the previous lesson. In this one, we will look into *hardware* and *software load balancing*.

Let's get on with it.

based load balancing.

Hardware load balancers

Hardware load balancers are highly performant physical hardware. They sit in front of the *application servers* and distribute the load based on the number of currently open connections to a server, compute utilization, and several other parameters.

Since these load balancers are physical hardware, they need maintenance and regular updates, just like any other server hardware would need. They are also expensive to set up compared to software load balancers, and their upkeep may require a certain skill set.

Also, the hardware load balancers have to be overprovisioned upfront to deal with peak traffic, which is not the case with software load balancers.

But when it comes to performance, hardware load balancers stand out.

If the business has network specialists and an IT team in-house, they can manage these load balancers. Otherwise, the developers are expected to wrap their heads around setting up these hardware load balancers with some assistance from the vendors. This is also the primary reason why developers prefer working with *software load balancers*.

Now, let's take a look at *software-based load balancing*.

Software load balancers

Software load balancers can be installed on *commodity hardware* and *VMs*. They are more cost-effective and offer more flexibility to the developers. *Software load balancers* can be upgraded and provisioned easily compared to hardware load balancers.

You will also find several *Load Balancers as a Service* (LBaaS) services online that enable you to directly plug in load balancers into your application

without you having to do any sort of setup.

Software load balancers are pretty advanced compared to *DNS load balancing*. They consider many parameters such as *data hosted by the servers, cookies, HTTP headers, CPU and memory utilization, load on the network*, etc., to route traffic across the servers.

They also continually perform health checks on the servers to keep an updated list of in-service machines.

Development teams prefer to work with software load balancers as hardware load balancers require specialists to manage them.

HAProxy is one example of a software load balancer widely used by the big guns in the industry, including *GitHub, Reddit, Instagram, AWS, Tumblr, StackOverflow*, to scale their systems.

Besides the round Robin algorithm, which *DNS Load balancers* use, software load balancers leverage several other algorithms to efficiently route traffic across the machines. Let's take a look.

Algorithms/Traffic routing approaches leveraged by load balancers

Round robin and weighted round robin

We know that the *round robin algorithm* sends *IP addresses* of machines sequentially to the clients. Parameters such as the server load, CPU consumption, and so on are not considered when sending the IP addresses to the clients.

We have another approach known as the *weighted round robin*, where based on the server's compute and traffic handling capacity, weights are assigned

to them. And then, based on server weights, traffic is routed to them using the *round robin algorithm*.

With this approach, more traffic is converged to machines that can handle a higher traffic load, thus efficiently using the resources.

This approach is pretty useful when the service is deployed across multiple data centers having different compute capacities. More traffic can be directed to the larger data centers containing more machines.

Least connections

When using this algorithm, the traffic is routed to the machine with the least open connections of all the machines in the cluster. There are two approaches to implement this.

In the first, it is assumed that all the requests will consume an equal amount of server resources, and the traffic is routed to the machine with the least open connections based on this assumption.

In this scenario, there is a possibility that the machine with the least open connections might already be processing requests demanding most of its *CPU* power. Routing more traffic to this machine would not be a good idea.

In the other approach, the *CPU* utilization and the request processing time of the chosen machine are also considered before routing the traffic to it. Machines with the shortest request processing time, most negligible CPU utilization, and the least open connections are suitable candidates to process future client requests.

The least connections approach comes in handy when the server has long opened connections like persistent connections in a gaming application.

Random

Following this approach, the traffic is randomly routed to the servers. The load balancer may also find similar servers in terms of existing load, request processing time, and so on. Then it randomly routes the traffic to these machines.

Hash

In this approach, the source IP where the request is coming from and the request URL are hashed to route the traffic to the backend servers.

Hashing the source IP ensures that a client's request with a certain IP will always be routed to the same server.

This facilitates a better user experience as the server has already processed the initial client requests and holds the client's data in its local memory. There is no need for it to fetch the client session data from the session memory of the cluster and process the request. This reduces latency.

Hashing the client IP also enables the client to re-establish the connection with the same server that was processing its request in case the connection drops.

Hashing a URL ensures that the request with that URL always hits a certain cache that already has data on it. This is to ensure that there is no cache miss.

This also averts the need for duplicating data in every cache and is, thus, a more efficient way to implement caching.

Well, this is pretty much it on the fundamentals of load balancing. In the next chapter, we will cover *monoliths* and *microservices*.

Back Mark As Completed Next

DNS Load Balancing

Load Balancing Quiz